

Les Fonctions en JavaScript (version simple pour débutant)

Les fonctions sont très importantes en JavaScript.

Une fonction est un bloc de code que l'on peut réutiliser plusieurs fois.

Au lieu d'écrire le même code encore et encore, on crée une fonction et on l'appelle quand on en a besoin.

Les fonctions servent à :

- organiser le programme ;
 - éviter les répétitions ;
 - donner un nom à une action ;
 - rendre le code plus facile à comprendre.
-

Définir une fonction

Créer une fonction signifie créer une action.

Exemple :

```
const square = function(x) {  
  return x * x;  
};
```

```
console.log(square(12));
```

Résultat :

144

Ici :

- **square** est le nom de la fonction ;
- **x** est le paramètre ;
- **return** renvoie le résultat.

Cette fonction calcule le carré d'un nombre.

Donc :

$12 \times 12 = 144$

Les paramètres

Les paramètres sont des valeurs que l'on donne à la fonction.

Exemple :

```
function greet(name) {  
  console.log("Bonjour " + name);  
}
```

```
greet("Jean");
```

Résultat :

Bonjour Jean

Ici :

- `name` est le paramètre ;
 - `"Jean"` est la valeur envoyée à la fonction.
-

Fonction sans paramètre

Une fonction peut aussi ne rien recevoir.

```
function makeNoise() {  
  console.log("Pling!");  
}
```

```
makeNoise();
```

Résultat :

Pling!

Le mot-clé return

`return` sert à renvoyer une valeur.

```
function add(a, b) {  
  return a + b;  
}
```

```
console.log(add(2, 3));
```

Résultat :

5

Sans **return**, la fonction ne renvoie rien.

Variables globales et locales

Variable globale

Une variable créée en dehors d'une fonction est globale.

```
let age = 20;
```

On peut l'utiliser partout dans le programme.

Variable locale

Une variable créée dans une fonction est locale.

```
function test() {  
  let name = "Paul";  
  console.log(name);  
}
```

name existe seulement dans cette fonction.

Les fonctions peuvent être dans d'autres fonctions

Exemple :

```
function outside() {  
  
  function inside() {  
    console.log("Je suis à l'intérieur");  
  }  
  
  inside();  
}  
  
outside();
```

Une fonction peut contenir une autre fonction.

Les fonctions comme valeurs

En JavaScript, une fonction est aussi une valeur.

On peut la stocker dans une variable.

```
let sayHello = function() {  
  console.log("Bonjour");  
};  
  
sayHello();
```

Déclaration de fonction

Il existe une manière plus simple d'écrire une fonction :

```
function square(x) {  
  return x * x;  
}
```

C'est la forme la plus utilisée par les débutants.

Fonctions fléchées (Arrow Functions)

Nouvelle façon moderne d'écrire les fonctions.

```
const add = (a, b) => {  
  return a + b;  
};
```

Version courte :

```
const add = (a, b) => a + b;
```

Les deux font la même chose.

Appel de fonction

Pour utiliser une fonction, on l'appelle.

```
greet("Marie");
```

Le nom suivi de parenthèses appelle la fonction.

La pile d'appels (Call Stack)

Quand une fonction appelle une autre fonction, JavaScript garde l'ordre des appels dans une pile.

Exemple :

```
function greet() {  
  console.log("Bonjour");  
}
```

```
greet();
```

JavaScript :

1. entre dans **greet**
 2. exécute le code
 3. revient au programme principal
-

Arguments optionnels

JavaScript accepte plus ou moins d'arguments que prévu.

```
function square(x) {  
  return x * x;  
}
```

```
console.log(square(4, true, "hello"));
```

Résultat :

16

Les arguments supplémentaires sont ignorés.

Valeur par défaut

On peut donner une valeur par défaut à un paramètre.

```
function greet(name = "ami") {  
  console.log("Bonjour " + name);  
}
```

```
greet();
```

Résultat :

Bonjour ami

Closure (Fermeture)

Une fonction peut “se souvenir” des variables autour d’elle.

Exemple :

```
function multiplier(factor) {  
  return number => number * factor;  
}
```

```
let double = multiplier(2);
```

```
console.log(double(5));
```

Résultat :

10

Ici :

- `factor` vaut 2
 - la fonction s'en souvient même après la fin de `multiplier`
-

Récursion

La récursion signifie qu'une fonction s'appelle elle-même.

Exemple :

```
function power(base, exponent) {  
  
  if (exponent == 0) {  
    return 1;  
  }  
  
  return base * power(base, exponent - 1);  
}  
  
console.log(power(2, 3));
```

Résultat :

8

Car :

$2 \times 2 \times 2 = 8$

Résumé simple

Une fonction sert à :

- réutiliser du code ;
 - organiser le programme ;
 - éviter les répétitions.
-

Les choses importantes à retenir

Élément	Rôle
<code>function</code>	crée une fonction
paramètre	valeur reçue
<code>return</code>	renvoie un résultat
appel de fonction	exécute la fonction
variable locale	existe seulement dans la fonction
arrow function	manière moderne d'écrire une fonction
réursion	fonction qui s'appelle elle-même